

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5

8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I Kanish Kumar V (192524361) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kanish Kumar V

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct	Limited or inconsistent use of tools	Minimal or incorrect use of tools

			implementatio n		
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalis m	5%	Highly professional, well- documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Recommended File Organization

For an e-commerce company storing 10 million product records, **B+ Tree indexed file organization** is recommended. It provides fast searching and efficient range queries, making it suitable for large databases.

Cost Analysis

Without an index, searching may require scanning millions of records. With a B+ Tree index, searching takes approximately **$O(\log N)$** time, reducing disk I/O. Although indexes require additional storage, the improved search performance makes the cost worthwhile.

```
mysql> CREATE DATABASE ecommerce;
Query OK, 1 row affected (0.01 sec)

mysql> USE ecommerce;
Database changed
mysql>
mysql> CREATE TABLE products (
  ->     product_id INT PRIMARY KEY,
  ->     product_name VARCHAR(100),
  ->     category_id INT,
  ->     price DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO products VALUES
  -> (101, 'iPhone 15', 1, 69999),
  -> (102, 'Galaxy S24', 1, 64999),
  -> (103, 'MacBook Air', 2, 99999),
  -> (104, 'Dell Inspiron', 2, 65999),
  -> (105, 'Sony Headphones', 3, 8999);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE INDEX idx_category ON products(category_id);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE INDEX idx_price ON products(price);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM products
  -> WHERE price BETWEEN 50000 AND 70000;
+-----+-----+-----+-----+
| product_id | product_name | category_id | price |
+-----+-----+-----+-----+
|          102 | Galaxy S24 |          1 | 64999.00 |
|          104 | Dell Inspiron |          2 | 65999.00 |
|          101 | iPhone 15 |          1 | 69999.00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Conclusion

B+ Tree indexing is suitable for 10 million product records because it provides fast searching, efficient range queries, and reduces disk I/O.

2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Indexing Strategy

For a banking system, a B+ Tree index is suitable because transactions need to be retrieved quickly using an account number and a date range.

Composite Index

A composite B+ Tree index can be created on account number and transaction date. Placing `account_no` first allows the database to quickly locate an account, while `transaction_date` supports efficient range searching.

```
mysql> CREATE TABLE transactions (  
->     transaction_id INT PRIMARY KEY,  
->     account_no INT,  
->     transaction_date DATE,  
->     amount DECIMAL(10,2),  
->     transaction_type VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE INDEX idx_account_date  
-> ON transactions(account_no, transaction_date);  
Query OK, 0 rows affected (0.03 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> SELECT *  
-> FROM transactions  
-> WHERE account_no = 1001  
-> AND transaction_date BETWEEN '2026-01-01' AND '2026-03-31';
```

Cost Analysis

Without an index, MySQL may scan all transaction records. With the composite B+ Tree index, it can directly locate the required account and then search only the specified date range, reducing disk I/O and improving retrieval speed.

Conclusion

The **composite B+ Tree index (account_no, transaction_date)** is the most appropriate strategy because it efficiently supports both account-based searches and date-range queries.

3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Multi-Level Indexing

A healthcare system can use multi-level B+ Tree indexing to provide fast access to patient records using both PatientID and DoctorName. A primary index can be created on PatientID, while a secondary index can be created on DoctorName.

```
mysql> CREATE TABLE patients (  
-> PatientID INT PRIMARY KEY,  
-> PatientName VARCHAR(100),  
-> DoctorName VARCHAR(100),  
-> Disease VARCHAR(100),  
-> AdmissionDate DATE  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> CREATE INDEX idx_doctor  
-> ON patients(DoctorName);  
Query OK, 0 rows affected (0.02 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> SELECT *  
-> FROM patients  
-> WHERE PatientID = 101;  
Empty set (0.00 sec)  
  
mysql>  
mysql> SELECT *  
-> FROM patients  
-> WHERE DoctorName = 'Dr. Kumar';
```

Cost Analysis

The primary index provides fast retrieval using PatientID, while the secondary index allows efficient searching by DoctorName. This avoids scanning the entire patient table and reduces disk I/O.

Conclusion

Using primary and secondary B+ Tree indexes provides efficient multi-level access to patient records through both PatientID and DoctorName.

4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Suitable Index Structure

For a logistics company with frequent insertions, deletions, and searches, a B+ Tree is more suitable than a B-Tree. B+ Trees keep all actual records at the leaf level and maintain linked leaf nodes, making them efficient for sequential and range-based shipment searches.

Comparison

B-Tree: Stores keys and records in both internal and leaf nodes. Searching can finish at any level, but sequential and range access is less efficient.

B+ Tree: Stores records only in leaf nodes, while internal nodes contain keys for navigation. Leaf nodes are linked, providing efficient range searches and sequential access.

```
mysql> CREATE TABLE shipments (
->     shipment_id INT PRIMARY KEY,
->     customer_name VARCHAR(100),
->     shipment_date DATE,
->     status VARCHAR(30)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE INDEX idx_shipment_date
-> ON shipments(shipment_date);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO shipments VALUES
-> (101, 'Arun', '2026-08-20', 'Delivered');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> DELETE FROM shipments
-> WHERE shipment_id = 101;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT *
-> FROM shipments
-> WHERE shipment_date BETWEEN '2026-08-01' AND '2026-08-31';
```

Cost Analysis

Both B-Tree and B+ Tree provide approximately $O(\log N)$ search, insertion, and deletion. However, B+ Tree is more efficient for range queries and sequential access because its leaf nodes are linked.

Conclusion

B+ Tree is more suitable for the logistics company because it supports dynamic insertion and deletion efficiently while providing better performance for shipment range searches and sequential processing.

5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Hashing Scheme

For an HR system with 5,000 employees, use EmployeeID as the search key. A hash function such as $\text{EmployeeID} \% 100$ can distribute records into 100 buckets.

Static Hashing

In static hashing, the number of buckets remains fixed. For 5,000 employees and 100 buckets, each bucket stores approximately 50 records.

```
mysql> CREATE TABLE employees (
  ->     EmployeeID INT PRIMARY KEY,
  ->     EmployeeName VARCHAR(100),
  ->     Department VARCHAR(50),
  ->     Salary DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE INDEX idx_employee_id
  -> ON employees(EmployeeID);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT *
  -> FROM employees
  -> WHERE EmployeeID = 1025;
```

Comparison

Feature	Static Hashing	Extendible Hashing
Bucket size	Fixed	Dynamic
Overflow handling	Overflow buckets	Bucket splitting
Performance	Good for stable data	Good for growing data
Implementation	Simple	More complex
Suitable for HR	Moderate	Better

Conclusion

For an HR system with **5,000 employees**, **static hashing** is sufficient if the number of employees remains relatively stable. However, **extendible hashing is more suitable** if the organization expects continuous employee growth because it dynamically handles bucket overflow without requiring a complete reorganization.

6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Combined Index Structure

For an airline reservation system, a combined B+ Tree indexing strategy is suitable. Use one index on FlightNumber for fast point queries and another index on FlightDate for efficient range queries.

```
mysql> CREATE TABLE flights (  
->     FlightNumber VARCHAR(10) PRIMARY KEY,  
->     FlightDate DATE,  
->     Source VARCHAR(50),  
->     Destination VARCHAR(50),  
->     SeatsAvailable INT  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql>  
mysql> CREATE INDEX idx_flight_date  
-> ON flights(FlightDate);  
Query OK, 0 rows affected (0.03 sec)  
Records: 0  Duplicates: 0  Warnings: 0  
  
mysql>  
mysql> SELECT *  
-> FROM flights  
-> WHERE FlightNumber = 'AI101';  
Empty set (0.00 sec)  
  
mysql>  
mysql> SELECT *  
-> FROM flights  
-> WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-31';
```

Cost Analysis

The FlightNumber index provides fast **point lookup**, while the FlightDate index efficiently handles **date-range searches**. Both provide approximately $O(\log N)$ search performance and reduce the need for full table scans.

Conclusion

Using **two B+ Tree indexes**, one on FlightNumber and one on FlightDate, provides efficient support for both point and range queries in the airline reservation system.

7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

File Organization

For a university student database, heap file organization can be used to store student records because records can be inserted efficiently. Secondary B+ Tree indexes can then be created for frequently searched fields.

Secondary Indexing

Create secondary indexes on Department and CGPA. The department index supports fast department-based queries, while the CGPA index supports efficient range queries.

```
mysql>
mysql> SELECT *
  -> FROM flights
  -> WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-31';
Empty set (0.00 sec)

mysql> CREATE TABLE students (
  ->     StudentID INT PRIMARY KEY,
  ->     StudentName VARCHAR(100),
  ->     Department VARCHAR(50),
  ->     CGPA DECIMAL(3,2)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> CREATE INDEX idx_department
  -> ON students(Department);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE INDEX idx_cgpa
  -> ON students(CGPA);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT *
  -> FROM students
  -> WHERE Department = 'CSE';
Empty set (0.00 sec)

mysql>
mysql> SELECT *
  -> FROM students
  -> WHERE CGPA BETWEEN 8.00 AND 9.00;
```

Cost Analysis

Without indexes, the database may scan all student records. Secondary B+ Tree indexes reduce the search cost to approximately $O(\log N)$ and make department and CGPA range queries faster. The indexes require additional storage but significantly reduce query time.

Conclusion

Heap file organization with secondary B+ Tree indexes on Department and CGPA is suitable because it provides efficient record insertion and fast department and CGPA range searches.

8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Given Data

- Number of records = 100,000
- Record size = 50 bytes
- Block size = 4 KB = 4096 bytes

Records per Block

$$\text{Records per block} = \left\lfloor \frac{4096}{50} \right\rfloor = 81$$

Therefore, each block can store **81 records**.

Heap File Organization

Heap files store records in no particular order.

$$\text{Blocks required} = \left\lceil \frac{100000}{81} \right\rceil = 1235$$

Block utilization:

$$\frac{100000 \times 50}{1235 \times 4096} \times 100 \approx 98.85\%$$

Sorted File Organization

Sorted files store records in sorted order. Assuming no extra space is reserved for future insertions:

$$\text{Blocks required} = 1235$$

Block utilization is approximately **98.85%**.

The disadvantage is that insertions and deletions may require shifting records or reorganizing blocks.

Hashed File Organization

In hashed organization, records are distributed among buckets using a hash function. Assuming an even distribution and no overflow:

$$\text{Blocks required} = 1235$$

The theoretical utilization is approximately **98.85%**. In practice, utilization can be lower because of **bucket overflow and unused space**.

Conclusion

All three organizations require approximately 1235 blocks when there is no additional overhead or overflow. Heap and Sorted files provide high block utilization, while Hashed files may have slightly lower practical utilization due to bucket overflow and unused space.

9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy for the platform

Composite Indexing Strategy

For a social media platform with 500 million posts, use B+ Tree indexes with user_id and timestamp for fast user-based and time-range queries. A separate index on hashtag can support hashtag searches.

Index Design

The main composite index should be:

(user_id, timestamp)

This allows the system to quickly find posts belonging to a particular user and then retrieve them within a timestamp range.

```
mysql> CREATE TABLE posts (
  ->     post_id BIGINT PRIMARY KEY,
  ->     user_id BIGINT,
  ->     timestamp DATETIME,
  ->     hashtag VARCHAR(100),
  ->     content TEXT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE INDEX idx_user_time
  -> ON posts(user_id, timestamp);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE INDEX idx_hashtag
  -> ON posts(hashtag);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> -- Posts of a particular user within a date range
mysql> SELECT *
  -> FROM posts
  -> WHERE user_id = 1001
  -> AND timestamp BETWEEN '2026-08-01' AND '2026-08-31';
Empty set (0.00 sec)

mysql>
mysql> -- Posts with a particular hashtag
mysql> SELECT *
  -> FROM posts
  -> WHERE hashtag = '#technology';
```

Cost Analysis

The (user_id, timestamp) composite B+ Tree index provides approximately $O(\log N)$ lookup and efficient timestamp range scanning. The hashtag index provides fast hashtag searches. Although maintaining indexes on **500 million records** requires significant storage, the reduction in disk I/O makes the indexes worthwhile.

Conclusion

A composite B+ Tree index on (user_id, timestamp) combined with a secondary index on hashtag is suitable for efficiently handling large-scale social media queries.

10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Performance Comparison

For a supermarket billing system handling 1 million transactions per day, the three file organizations have different performance characteristics.

Heap File

Records are stored in no particular order. Insertion is very fast because a new transaction can be added at the end of the file. However, searching requires a linear scan, so search and deletion are slow.

Sorted File

Records are maintained in sorted order. Search is fast using binary search, but insertion and deletion are expensive because records may need to be shifted to maintain the order.

Hashed File

Records are distributed into buckets using a hash function. Search, insertion, and deletion are fast on average, making hashing highly suitable for transaction lookup using a unique transaction ID.

```
mysql> CREATE TABLE transactions (
->     transaction_id BIGINT PRIMARY KEY,
->     customer_id INT,
->     transaction_date DATE,
->     amount DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO transactions VALUES
-> (1001, 501, '2026-08-23', 2500.00);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SELECT *
-> FROM transactions
-> WHERE transaction_id = 1001;
+-----+-----+-----+-----+
| transaction_id | customer_id | transaction_date | amount |
+-----+-----+-----+-----+
|          1001 |          501 | 2026-08-23      | 2500.00 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> DELETE FROM transactions
-> WHERE transaction_id = 1001;
Query OK, 1 row affected (0.01 sec)
```

Conclusion

For a supermarket billing system with **1 million daily transactions**, **Hashed File Organization is the most suitable** when transactions are mainly accessed using `transaction_id`. Heap files are best for fast insertion, while sorted files are preferable when ordered or range-based access is frequently required.